# 3.4: Analog Input and Output (I/O)

## Introduction to Microcontrollers with Arduino

This module introduces the fundamental concepts of Analog Input and Output (I/O) using Arduino microcontrollers. We will explore how to read analog sensor data using the `analogRead()` function and how to generate analog-like output signals using Pulse Width Modulation (PWM) with the `analogWrite()` function. These functions are crucial for interacting with the physical world, enabling your Arduino projects to respond to environmental changes and control various devices.

## 1. Analog Input with `analogRead()`

Microcontrollers are inherently digital devices, meaning they primarily understand ON/OFF states (0s and 1s). However, the real world is analog, with continuously varying signals like temperature, light intensity, and sound. To bridge this gap, Arduino boards are equipped with an Analog-to-Digital Converter (ADC).

### 1.1. The Analog-to-Digital Converter (ADC)

An **Analog-to-Digital Converter (ADC)** is an electronic circuit that converts a continuous analog voltage into a discrete digital value. Most Arduino boards, such as the Arduino Uno, feature a 10-bit ADC. This means it can represent analog voltages as one of $2^{10} = 1024$ discrete levels.

- **Input Voltage Range**: Typically 0V to 5V (or 0V to 3.3V, depending on the board's operating voltage).
- **Digital Output Range**: 0 to 1023.
- **Resolution**: The smallest change in voltage that the ADC can detect. For a 5V system with a 10-bit ADC, the resolution is:

$$\text{Resolution} = \frac{\text{Reference Voltage}}{\text{Number of Steps}} = \frac{5V}{1024} \approx 0.00488V \text{ or } 4.88mV \text{ per unit}$$

## 1.2. The `analogRead()` Function

The `analogRead()` function reads the voltage on a specified analog input pin and converts it into a digital value. This value is an integer ranging from 0 to 1023.

**Syntax:**

```
int analogRead(pin);
```

- `pin`: The analog input pin to read from (e.g., `A0`, `A1`, etc.).
- **Returns**: An integer value from 0 to 1023, representing the analog voltage.

**Important Note**: Unlike digital pins, analog input pins do not require `pinMode()` to be set as `INPUT` before use.

## 1.3. Reading Potentiometers

A **potentiometer** is a three-terminal resistor with a rotating contact that acts as a voltage divider. By rotating the knob, you can change the resistance and thus the voltage at the wiper (middle) pin. This makes it ideal for controlling parameters like volume, brightness, or motor speed.

**Circuit Connection:**

- One outer pin to 5V (or 3.3V).
- The other outer pin to GND.
- The middle (wiper) pin to an Arduino analog input pin (e.g., `A0`).

**Example Code: Reading a Potentiometer**

This example reads the value from a potentiometer connected to analog pin `A0` and prints it to the Serial Monitor.

```
const int potentiometerPin = A0; // Potentiometer connected to analog pin A0
int sensorValue = 0;             // Variable to store the value read from
the potentiometer

void setup() {
  Serial.begin(9600); // Initialize serial communication at 9600 bits per
second
}

void loop() {
  sensorValue = analogRead(potentiometerPin); // Read the analog value from
the potentiometer
  Serial.print("Potentiometer Value: ");
  Serial.println(sensorValue); // Print the value to the Serial Monitor
  delay(100); // Wait for 100 milliseconds before the next reading
}
```

## 1.4. Reading Photoresistors (LDRs)

A **photoresistor**, also known as a Light-Dependent Resistor (LDR), is a passive component whose resistance decreases with increasing light intensity. This property makes it useful for detecting light levels.

**Circuit Connection (Voltage Divider):**

To read an LDR with Arduino, it's typically used in a voltage divider configuration:

- One end of the LDR to 5V (or 3.3V).
- The other end of the LDR to an analog input pin (e.g., `A0`).
- A fixed resistor (e.g., 10kΩ) from the analog input pin to GND.

**Example Code: Reading a Photoresistor**

This example reads the light level from a photoresistor connected to analog pin `A0` and prints it to the Serial Monitor.

```
const int photoresistorPin = A0; // Photoresistor connected to analog pin A0
int lightLevel = 0;              // Variable to store the light level

void setup() {
  Serial.begin(9600); // Initialize serial communication
}

void loop() {
  lightLevel = analogRead(photoresistorPin); // Read the analog value from
the photoresistor
  Serial.print("Light Level: ");
  Serial.println(lightLevel); // Print the light level to the Serial Monitor
  delay(100); // Wait for 100 milliseconds
}
```

## 2. Analog Output with `analogWrite()` (PWM)

While Arduino's digital pins can only output HIGH (5V) or LOW (0V), we can simulate analog output using a technique called **Pulse Width Modulation (PWM)**. PWM allows us to control the average power delivered to a device by rapidly switching a digital signal ON and OFF.

### 2.1. Pulse Width Modulation (PWM)

**Pulse Width Modulation (PWM)** is a technique where the width of a digital pulse is varied to control the average power delivered to a load. By changing the **duty cycle** (the ratio of ON time to the total period), we can effectively control the average voltage or current. Arduino's `analogWrite()` function uses PWM.

- **Frequency**: The rate at which the pulse switches ON and OFF. For most Arduino boards, the PWM frequency is around 490 Hz or 980 Hz, depending on the pin.
- **Duty Cycle**: The percentage of one period in which the signal is ON. A 0% duty cycle means the signal is always OFF, and a 100% duty cycle means the signal is always ON.

## 2.2. The `analogWrite()` Function

The `analogWrite()` function writes an analog value (PWM wave) to a pin. This function is used to generate a square wave with a specified duty cycle, effectively simulating an analog output.

**Syntax:**

```
analogWrite(pin, value);
```

- `pin`: The Arduino pin to output the PWM signal. On most Arduino boards, only specific digital pins support PWM (often marked with a `~` symbol, e.g., pins 3, 5, 6, 9, 10, 11 on an Uno).
- `value`: The duty cycle, an integer between 0 (0% duty cycle, always OFF) and 255 (100% duty cycle, always ON).

**Important Note**: You do not need to call `pinMode()` to set the pin as an output before calling `analogWrite()`. The `analogWrite()` function has no relation to the analog input pins or the `analogRead()` function.

## 2.3. Dimming LEDs with PWM

By varying the `value` parameter in `analogWrite()`, we can control the brightness of an LED. A lower `value` results in a shorter ON time (lower duty cycle) and thus dimmer light, while a higher `value` results in a longer ON time (higher duty cycle) and brighter light.

**Circuit Connection:**

- Connect the long leg (anode) of an LED to a PWM-enabled digital pin on the Arduino (e.g., pin 9) via a current-limiting resistor (e.g., 220Ω).
- Connect the short leg (cathode) of the LED to GND.

**Example Code: Fading an LED**

This example demonstrates how to fade an LED in and out using `analogWrite()`.

```
const int ledPin = 9; // LED connected to PWM pin 9

void setup() {
  // No pinMode() needed for analogWrite()
}

void loop() {
  // Fade in
  for (int brightness = 0; brightness <= 255; brightness += 5) {
    analogWrite(ledPin, brightness);
    delay(30);
  }

  // Fade out
  for (int brightness = 255; brightness >= 0; brightness -= 5) {
    analogWrite(ledPin, brightness);
    delay(30);
  }
}
```

## 2.4. Motor Speed Control with PWM

PWM is also widely used to control the speed of DC motors. By varying the duty cycle of the PWM signal applied to a motor, we can control the average voltage supplied to it, thereby controlling its speed. A higher duty cycle results in higher average voltage and faster motor speed.

**Circuit Connection (using a motor driver):**

Directly connecting a motor to Arduino pins is generally not recommended due to current limitations. A motor driver (e.g., L298N H-bridge) is typically used.

- Connect the motor driver's input pin (for speed control) to a PWM-enabled digital pin on the Arduino (e.g., pin 5).
- Connect the motor driver's power and ground to an external power supply.
- Connect the motor to the motor driver's output terminals.

**Example Code: Controlling DC Motor Speed with Potentiometer**

This example uses a potentiometer to control the speed of a DC motor via PWM.

```
const int motorPin = 5;      // Motor speed control pin connected to PWM pin
5
const int potentiometerPin = A0; // Potentiometer connected to analog pin A0
int sensorValue = 0;             // Variable to store the potentiometer
value
int motorSpeed = 0;              // Variable to store the motor speed (0-
255)

void setup() {
  pinMode(motorPin, OUTPUT); // Set motor control pin as output
  Serial.begin(9600);        // Initialize serial communication
}

void loop() {
  sensorValue = analogRead(potentiometerPin); // Read potentiometer value
(0-1023)
  // Map the potentiometer value (0-1023) to analogWrite() range (0-255)
  motorSpeed = map(sensorValue, 0, 1023, 0, 255);
  analogWrite(motorPin, motorSpeed); // Set motor speed

  Serial.print("Potentiometer: ");
  Serial.print(sensorValue);
  Serial.print(", Motor Speed (PWM): ");
  Serial.println(motorSpeed);
  delay(50); // Small delay
}
```

## 3. Practice Problems

1. **Problem**: Design an Arduino sketch that reads the value from a photoresistor and controls the brightness of an LED. As the light level increases, the LED should become brighter.

   **Solution Hint**: Use `analogRead()` for the photoresistor and `analogWrite()` for the LED. Remember to map the `analogRead()` range (0-1023) to the `analogWrite()` range (0-255).

2. **Problem**: Write an Arduino sketch that uses two potentiometers. One potentiometer should control the speed of a DC motor, and the other should control the fade rate of an LED (how fast it fades in and out).

**Solution Hint**: You'll need two `analogRead()` calls and one `analogWrite()` for the motor. For the LED fade rate, the second potentiometer's value can be used to adjust the `delay()` in the fading loop.

## 4. Conclusion

Analog Input and Output are fundamental concepts in microcontroller programming, enabling interaction with the real world. `analogRead()` allows us to interpret continuous sensor data, while `analogWrite()` with PWM provides a means to control analog-like devices such as LEDs and motors. Mastering these functions opens up a vast array of possibilities for creating interactive and responsive Arduino projects.

## 5. References


- analogRead() - Arduino Documentation
- analogWrite() - Arduino Documentation
- Pulse Width Modulation (PWM) - Arduino Documentation
- Potentiometer Basics - Arduino Documentation
- How to Use a Photoresistor! - Arduino Project Hub